UNIVERSITATEA TEHNICĂ A MOLDOVEI

Технический университет Молдовы

Факультет вычислительной техники,
информатики и микроэлектроники

Департамент программной инженерии и автоматики

Программирование мобильных приложений

Programarea aplicațiilor mobile

# Отчёт по лабораторной работе № 3

*Навигация, формы и валидация*
*Индивидуальная тема RunBaza*

Выполнил: Hamuev Oleg

Группа: CR-232

Проверил: Leah A.

Учебный год 2026–2027

Кишинёв 2026

# 1 Цель и результат работы

В лабораторной работе 3 интерфейс RunBaza из Л2 дополнен маршрутизацией go_router, постоянной панелью вкладок и формами с проверкой ввода. Пользователь может выбрать забег, открыть дистанцию, заполнить параметры участия или поездки и вернуться к предыдущему экрану. Вход и выход меняют стек навигации.

RunBaza — индивидуальная тема: календарь забегов и поиск попутчиков в Молдове и Румынии. Разработка продолжается в Lab2/run_baza без создания нового Flutter-проекта. Каталог Lab3 содержит отчёт, руководство и доказательства проверки; версия Л2 сохранена в ветке lab/2-runbaza-ui.

## Выполненные требования

| Требование Л3 | Реализация |
|---|---|
| Единая карта маршрутов | lib/router.dart; MaterialApp.router; go_router 18.0.2 |
| Передача идентификатора | /races/:id и /transfers/:id; поиск объекта внутри экрана |
| Постоянные вкладки | StatefulShellRoute.indexedStack, пять StatefulShellBranch |
| Формы | Вход, участие, водитель, пассажир, профиль |
| Проверка полей | Form, GlobalKey<FormState>, validator, фильтры ввода |
| Обратная связь | SnackBar после успешной проверки, AlertDialog перед выходом |
| Проверка | 100 автоматических тестов и сценарий на iPhone Simulator |

## Границы этапа

Используются фиксированные mock-данные: 8 забегов, 6 поездок, 6 запросов и 6 записей плана. Формы не добавляют записи, не бронируют места и не отправляют данные в сеть. Вход принимает любые корректно заполненные поля и не является проверкой учётной записи. Хранение данных и настоящая авторизация относятся к следующим этапам.

## Визуальная основа

Изучен Samepace, main 438e942d9ebe0437840421ee5be6c604b75dfd86. В Flutter адаптированы тёплая светлая палитра, графитовая тема с лаймом, идея бегового паспорта и выбор темпа и видимости участия. Сохранены оригинальные векторные иллюстрации городов из Л2. Vue-компоненты и сервер Samepace не включены в мобильный проект.

# 2 Карта маршрутов

| Путь | Экран и назначение |
|---|---|
| /login | LoginScreen — вне оболочки с вкладками |
| /home | HomeScreen — ближайший старт и дистанции |
| /races | RaceIndexScreen — поиск и фильтр страны |
| /races/:id | RaceDetailsScreen — выбранный забег |
| /races/:id/distance/:km | DistanceScreen — дистанция и анимация |
| /races/:id/participate | ParticipationFormScreen — темп и видимость |
| /transfers | TransfersIndexScreen — выбор забега для поездки |
| /transfers/:id | TransferScreen — водители и пассажиры |
| /transfers/:id/offer | TripFormScreen(driver: true) — поездка |
| /transfers/:id/request | TripFormScreen(driver: false) — запрос места |
| /plan | MyPlanScreen — личный план |
| /profile | SettingsScreen — форма профиля и выход |

Каждая из пяти веток имеет собственный Navigator. Дочерние пути записаны без начального слеша. Вход расположен снаружи shell, поэтому на нём нет нижней панели. Неизвестный путь, ID или неподдерживаемая дистанция приводят на экран восстановления с кнопкой календаря.

### Разница между переходами

context.push добавляет детали или форму поверх текущего экрана; context.pop возвращает назад. context.go строит стек для указанного адреса: после входа используется /home, после выхода /login. Вкладки переключаются через navigationShell.goBranch, что сохраняет отдельные стеки и позиции прокрутки.

```
context.push('/races/${race.id}');
context.push('/transfers/$id/offer');
context.pop();
context.go('/home');
navigationShell.goBranch(index);
```

# 3 Полный код маршрутизатора

lib/router.dart, строки 1–40. Конфигурация соответствует исходному коду приложения.

```
import 'package:go_router/go_router.dart';

import 'widgets/app_shell.dart';

import 'screens/home_screen.dart';
import 'screens/trip_form_screen.dart';
import 'screens/participation_form_screen.dart';
import 'screens/login_screen.dart';
import 'screens/race_index_screen.dart';
import 'screens/race_details_screen.dart';
import 'screens/distance_screen.dart';
import 'screens/transfers_index_screen.dart';
import 'screens/transfer_screen.dart';
import 'screens/my_plan_screen.dart';
import 'screens/settings_screen.dart';
import 'screens/not_found_screen.dart';

GoRouter createRouter({String initialLocation = '/login'}) => GoRouter(
  initialLocation: initialLocation,
  errorBuilder: (context, state) => const NotFoundScreen(),
  routes: [
    GoRoute(path: '/login', builder: (context, state) => const LoginScreen()),
    StatefulShellRoute.indexedStack(
      builder: (context, state, navigationShell) =>
          AppShell(navigationShell: navigationShell),
      branches: [
        StatefulShellBranch(
          routes: [
            GoRoute(
              path: '/home',
              builder: (context, state) => const HomeScreen(),
            ),
          ],
        ),
        StatefulShellBranch(
          routes: [
            GoRoute(
              path: '/races',
              builder: (context, state) => const RaceIndexScreen(),
              routes: [
```

createRouter позволяет тесту выбрать начальный адрес. В приложении по умолчанию открывается /login. errorBuilder обрабатывает путь, отсутствующий в карте.

# 3 Полный код маршрутизатора продолжение

lib/router.dart, строки 41–80. Конфигурация соответствует исходному коду приложения.

```
              GoRoute(
                path: ':id',
                builder: (context, state) =>
                    RaceDetailsScreen(id: state.pathParameters['id']!),
                routes: [
                  GoRoute(
                    path: 'participate',
                    builder: (context, state) => ParticipationFormScreen(
                      id: state.pathParameters['id']!,
                    ),
                  ),
                  GoRoute(
                    path: 'distance/:km',
                    builder: (context, state) => DistanceScreen(
                      id: state.pathParameters['id']!,
                      distance:
                          double.tryParse(state.pathParameters['km']!) ?? -1,
                    ),
                  ),
                ],
              ),
            ],
          ),
        ],
      ),
      StatefulShellBranch(
        routes: [
          GoRoute(
            path: '/transfers',
            builder: (context, state) => const TransfersIndexScreen(),
            routes: [
              GoRoute(
                path: ':id',
                builder: (context, state) =>
                    TransferScreen(id: state.pathParameters['id']!),
                routes: [
                  GoRoute(
                    path: 'offer',
                    builder: (context, state) => TripFormScreen(
                      id: state.pathParameters['id']!,
```

state.pathParameters содержит строки из адреса. Double.tryParse используется для дистанции; неверное значение не вызывает исключение. Сам экран проверяет, что дистанция принадлежит забегу.

# 3 Полный код маршрутизатора продолжение

lib/router.dart, строки 81–116. Конфигурация соответствует исходному коду приложения.

```
                      driver: true,
                    ),
                  ),
                  GoRoute(
                    path: 'request',
                    builder: (context, state) => TripFormScreen(
                      id: state.pathParameters['id']!,
                      driver: false,
                    ),
                  ),
                ],
              ),
            ],
          ),
        ],
      ),
      StatefulShellBranch(
        routes: [
          GoRoute(
            path: '/plan',
            builder: (context, state) => const MyPlanScreen(),
          ),
        ],
      ),
      StatefulShellBranch(
        routes: [
          GoRoute(
            path: '/profile',
            builder: (context, state) => const SettingsScreen(),
          ),
        ],
      ),
    ],
  ),
],
);
```

Обе формы поездки используют один виджет с параметром driver. Маршрут передаёт только идентификатор забега и режим формы. Данные события получаются через findRace внутри экрана.

# 4 Постоянные вкладки

Нижняя панель принадлежит AppShell и остаётся видимой на вложенных экранах. Это один NavigationBar, а не отдельная панель на каждом экране. StatefulShellRoute хранит состояние веток в течение текущего сеанса приложения.

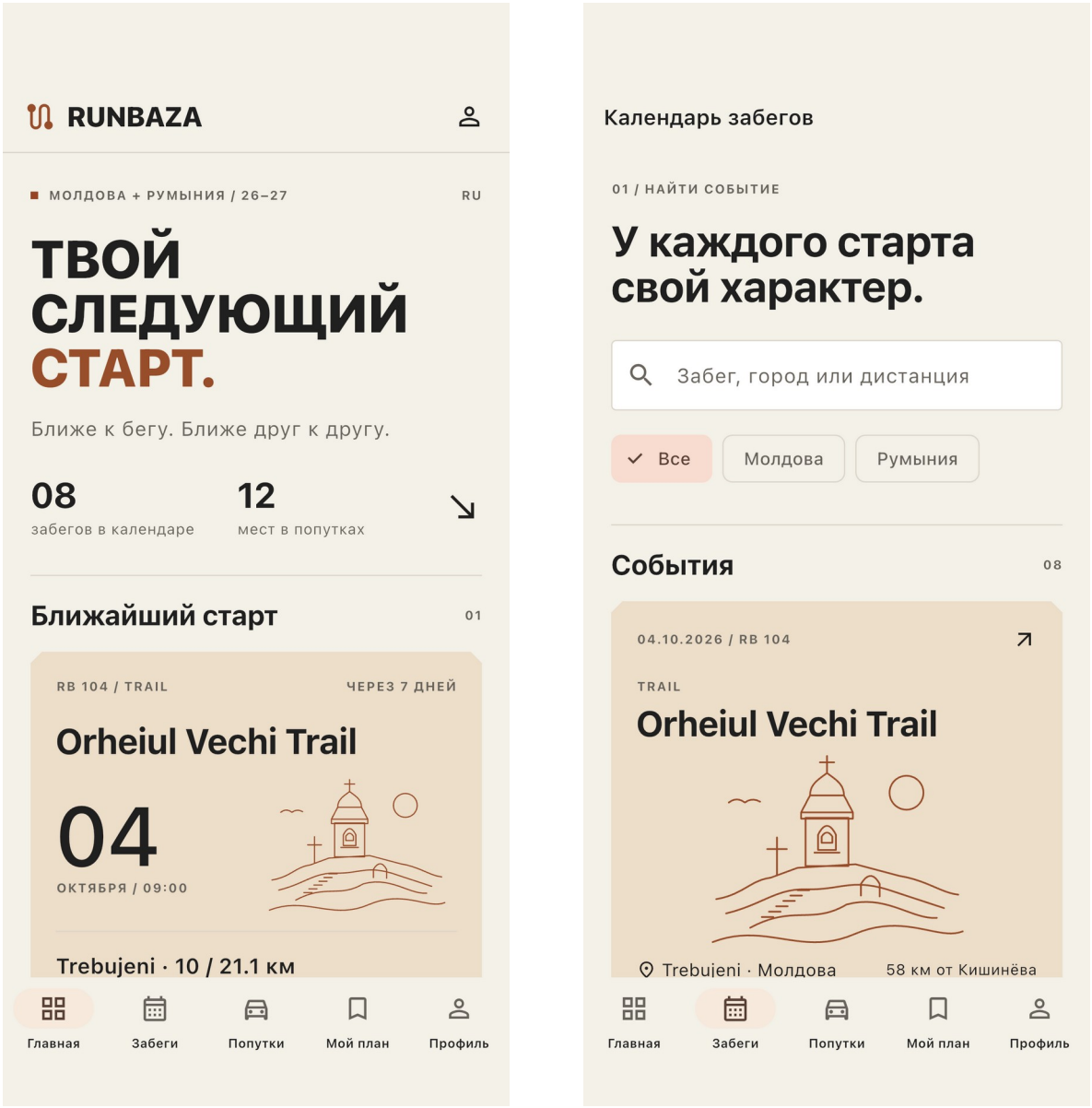



Рисунок 1. Ветки «Главная» и «Забеги» на iPhone 17 Pro Simulator.

Проверка: в календаре ввести Iaşi, открыть забег, перейти в профиль и вернуться. Возвращается тот же экран деталей; после «Назад» остаётся введённый поисковый запрос. Отдельный тест проверяет сохранение позиции прокрутки.

# 5 Разделы поездок и личного плана

Вкладка «Попутки» сначала предлагает выбрать событие, затем показывает относящиеся к нему поездки. Вкладка «Мой план» содержит список записей из mock-данных. Обе ветки используют существующий компонент RaceCard.

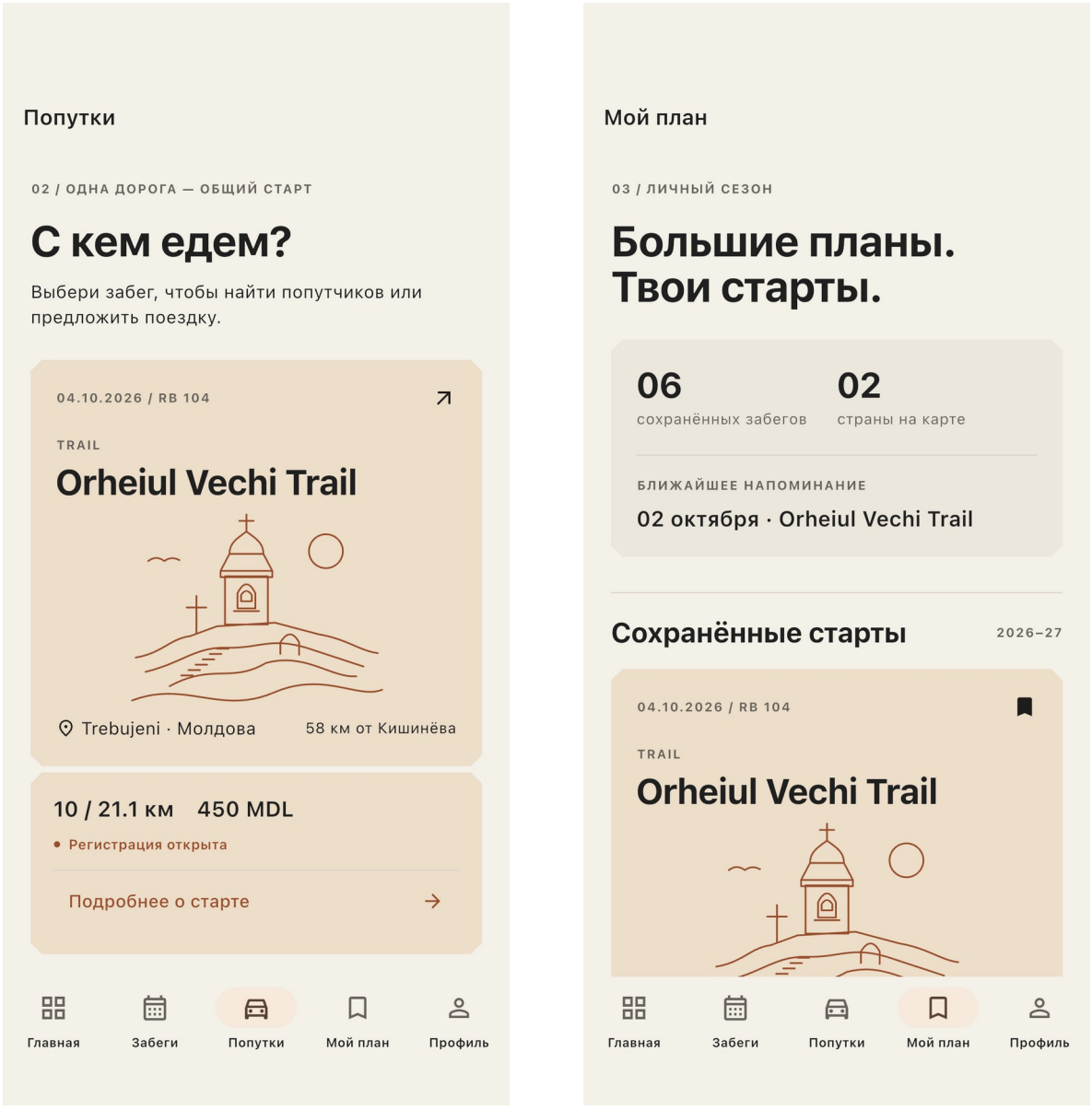



Рисунок 2. Третья и четвёртая ветки оболочки приложения.

Переход к карточке забега передаёт её строковый ID. Маршрутизатор не получает целый объект Race через extra. Это делает адрес однозначным и позволяет открыть тот же экран напрямую в тесте.

# 6 Разные идентификаторы и детали

Один RaceDetailsScreen обслуживает все события. Параметр id равен r1 для Orheiul Vechi Trail и r3 для Iaşi Autumn Run. В build выполняется findRace(id); название, география, цена и доступные дистанции берутся из найденного объекта.

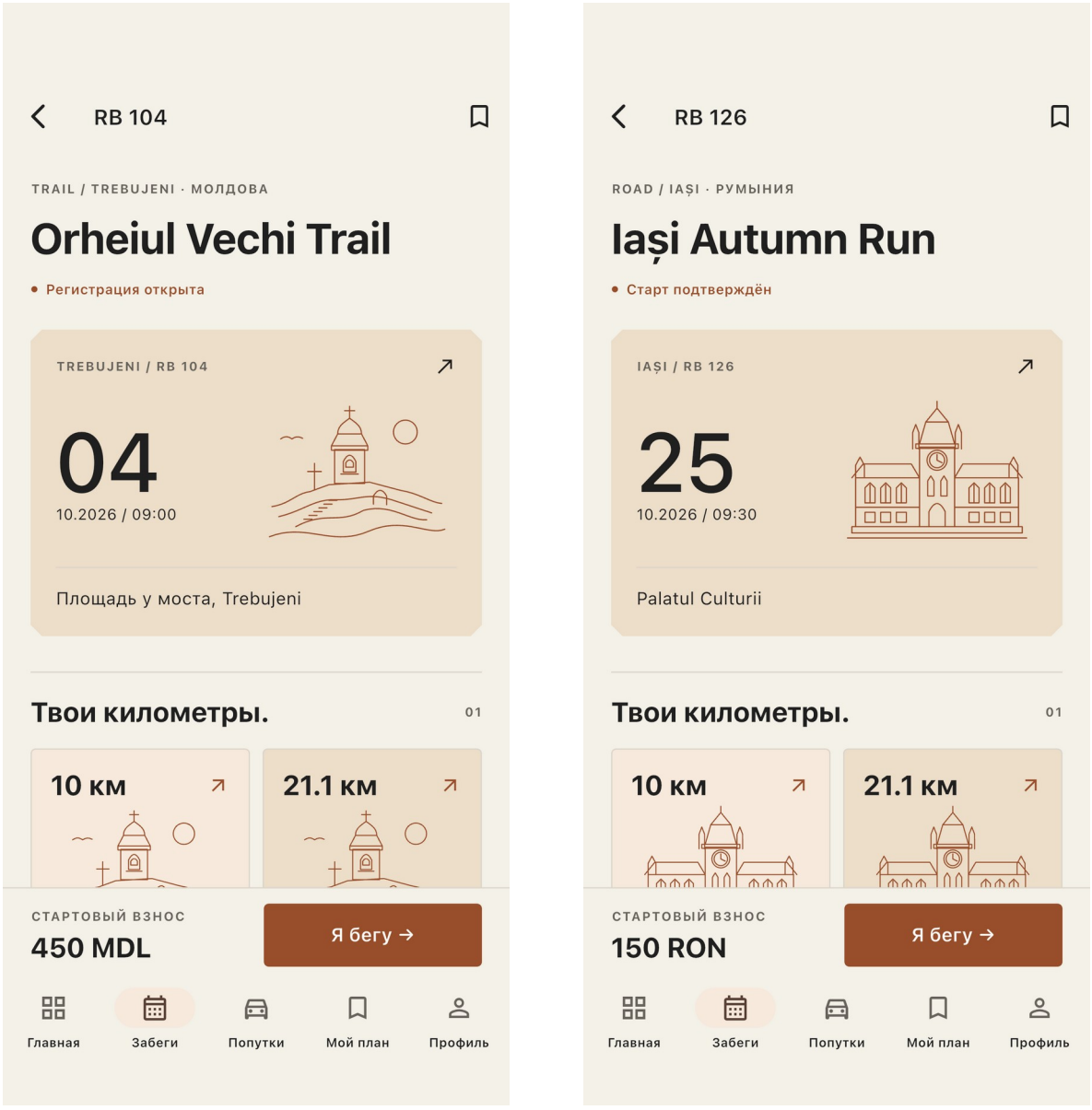



Рисунок 3. /races/r1 и /races/r3 открывают разные забеги.

Молдавские и румынские события сохраняют собственные иллюстрации и названия стран. Кнопка «Я бегу» ведёт к форме параметров участия; её подтверждение не является покупкой стартового слота.

# 7 Вход и проверка полей

LoginScreen преобразован в StatefulWidget с Form и двумя TextEditingController. E-mail проверяется на структуру адреса без пробелов, пароль — на длину от шести символов. Пароль скрыт; кнопка глаза меняет obscureText.

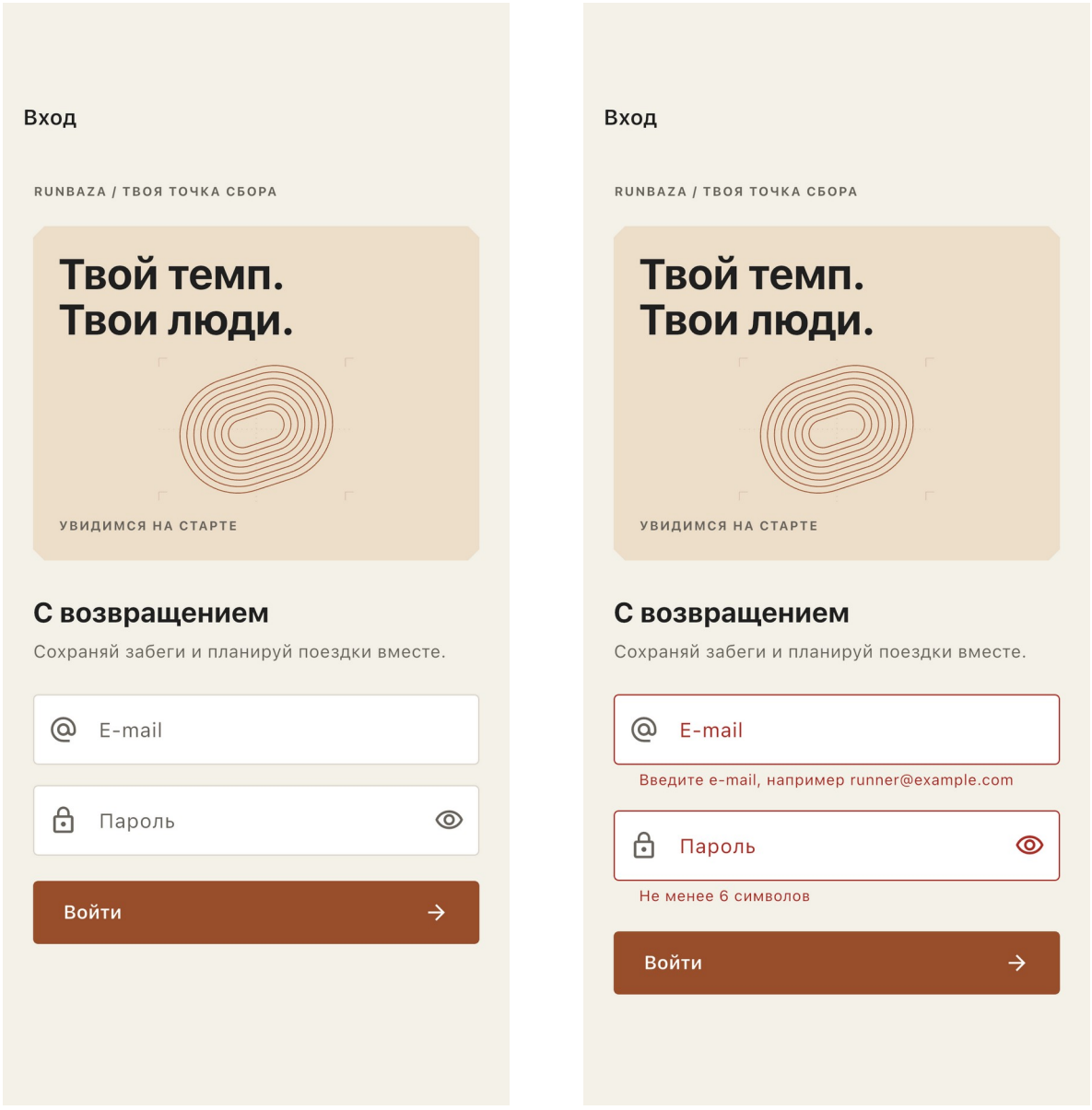



Рисунок 4. Пустая форма входа и результат отправки с ошибками.

При ошибке validate возвращает false, переход не выполняется. После корректного ввода вызывается context.go('/home'); /login исчезает из стека, поэтому кнопка «Назад» не возвращает форму входа. Проверка полей локальная, без серверной аутентификации.

# 8 Форма участия в забеге

Форма «Мой старт» продолжает идею Samepace: выбрать дистанцию события, комфортный темп и круг видимости. DropdownButtonFormField требует явного выбора. Темп проверяется как минуты и секунды на километр в диапазоне от 3:00 до 15:00.

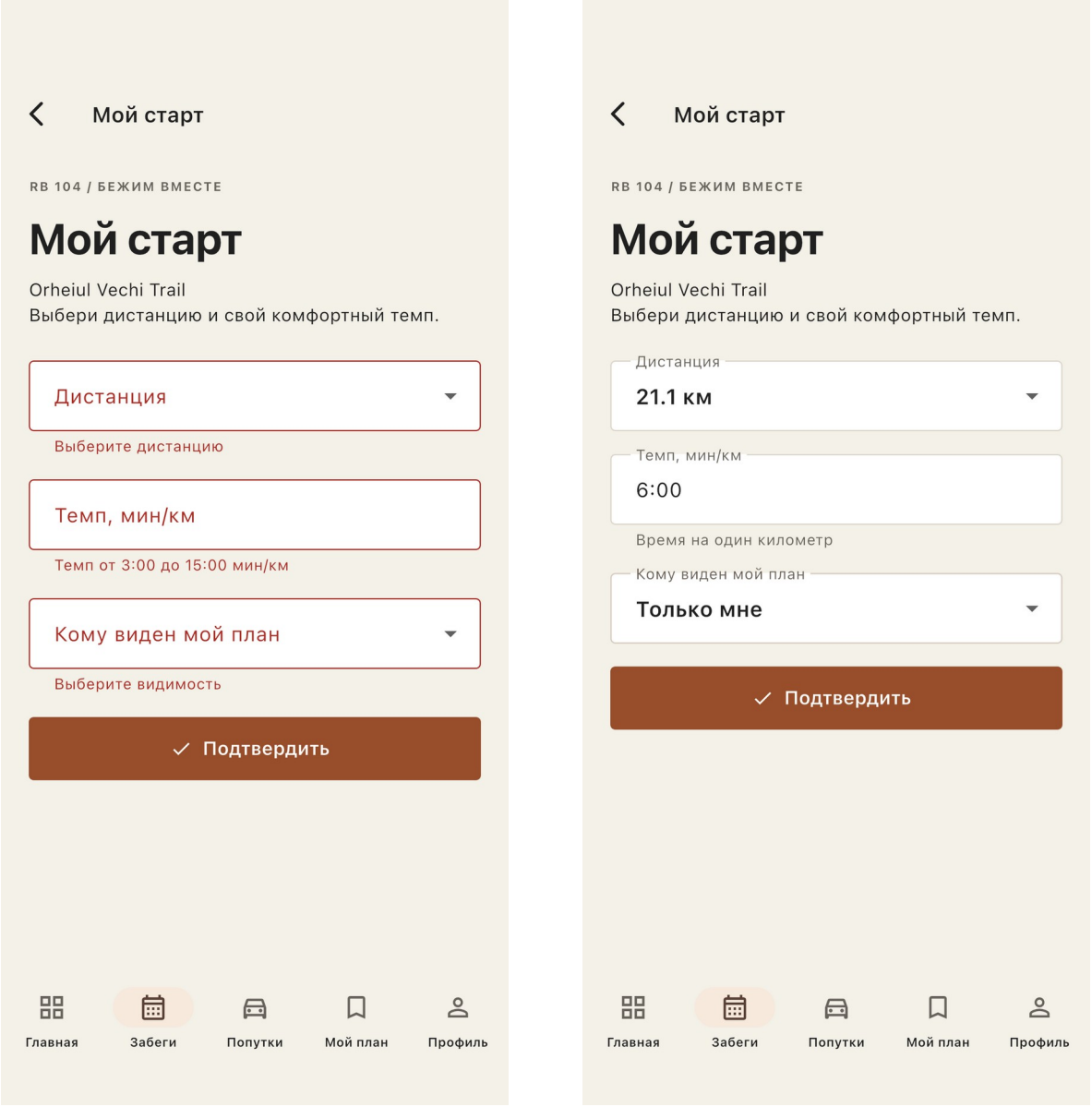



Рисунок 5. Ошибки обязательных полей и корректно заполненная форма.

Состояние полей хранится внутри StatefulWidget. Controller освобождается в dispose. После успешной проверки экран закрывается через pop, а SnackBar сообщает «Параметры старта проверены». Список плана не изменяется.

# 9 Формы водителя и пассажира

TripFormScreen использует режим driver. Для водителя дополнительно требуется вклад с человека в MDL. Общие поля: имя, город, место встречи, время, количество мест и телефон. Фильтры ограничивают ввод символов, а validator проверяет смысл и диапазон.

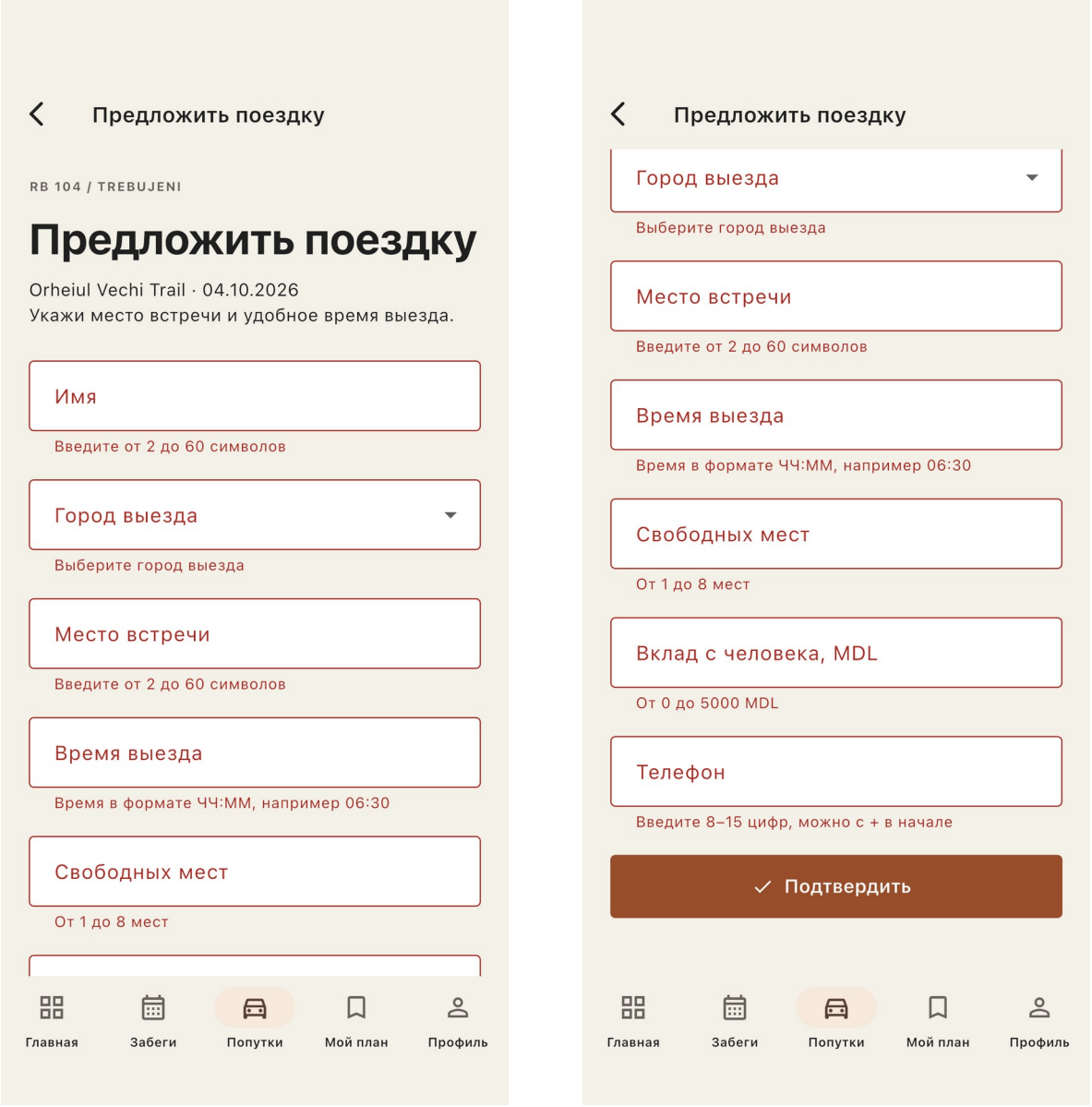



Рисунок 6. Верхняя и нижняя части формы водителя с ошибками.

Например, цифровая клавиатура не делает число допустимым: 0 мест отклоняется, а 0 MDL разрешает бесплатную поездку. Время 24:00 недопустимо; 23:59 корректно. Проверяются также пустые поля и вставка текста.

# 10 Запрос места и профиль

Форма пассажира не содержит цену: пользователь указывает нужное количество мест и контакт. Вкладка профиля содержит беговой паспорт и редактируемые поля имени, города и темпа. Введённые данные остаются локальными до закрытия экрана или сеанса.

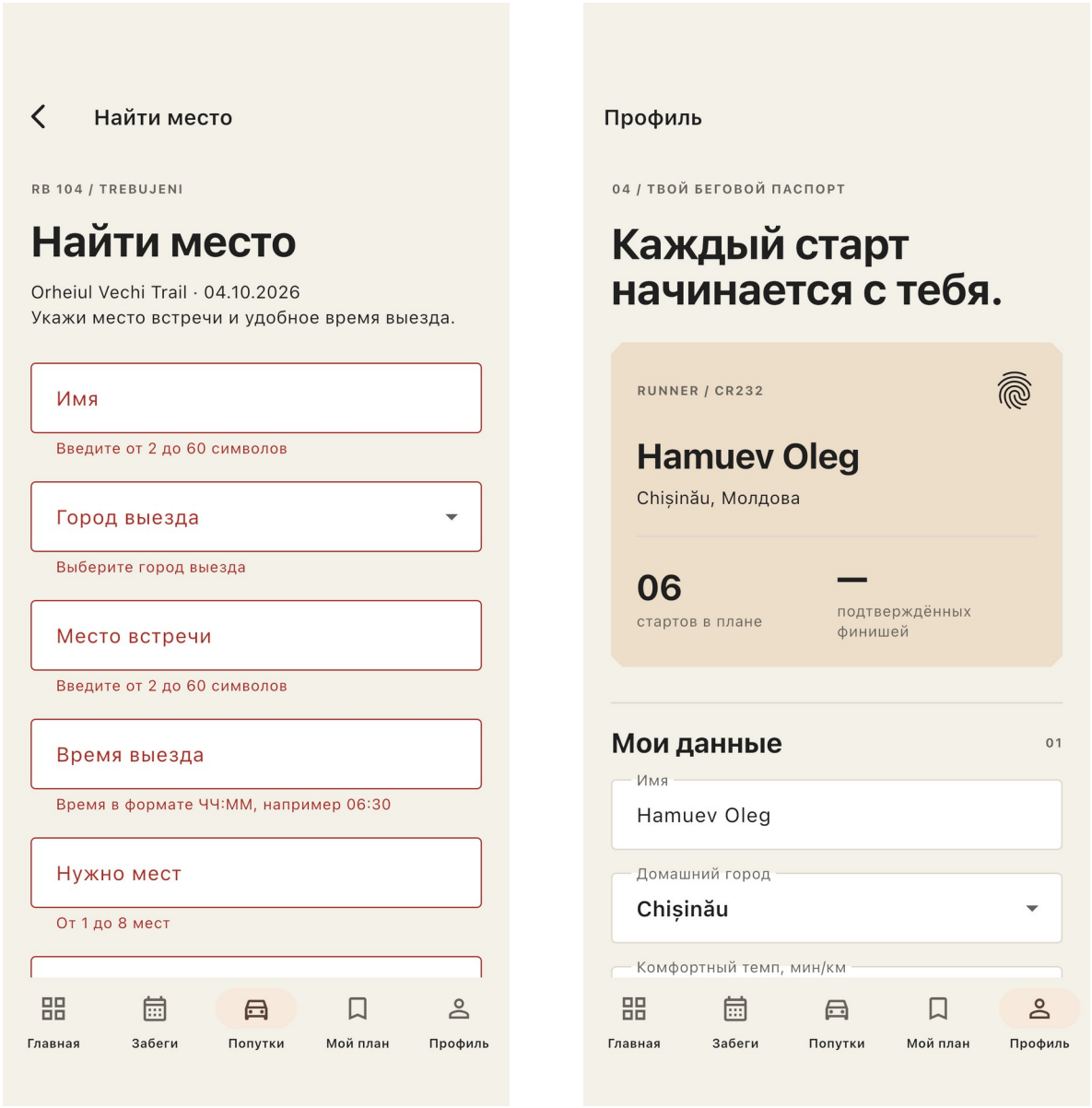



Рисунок 7. Валидация заявки пассажира и пятая ветка «Профиль».

Статистика паспорта берётся из исходных данных. Подтверждённые финиши не выдумываются. Сохранение новых значений в карточку или базу данных на этом этапе не реализуется; кнопка проверяет заполнение формы.

# 11 Проверка профиля и ошибочного адреса

Проверка имени и темпа выполняется и в профиле. Ошибка удерживает пользователя на форме. Неизвестный идентификатор обрабатывается отдельно: отсутствие записи не превращается в исключение firstWhere.

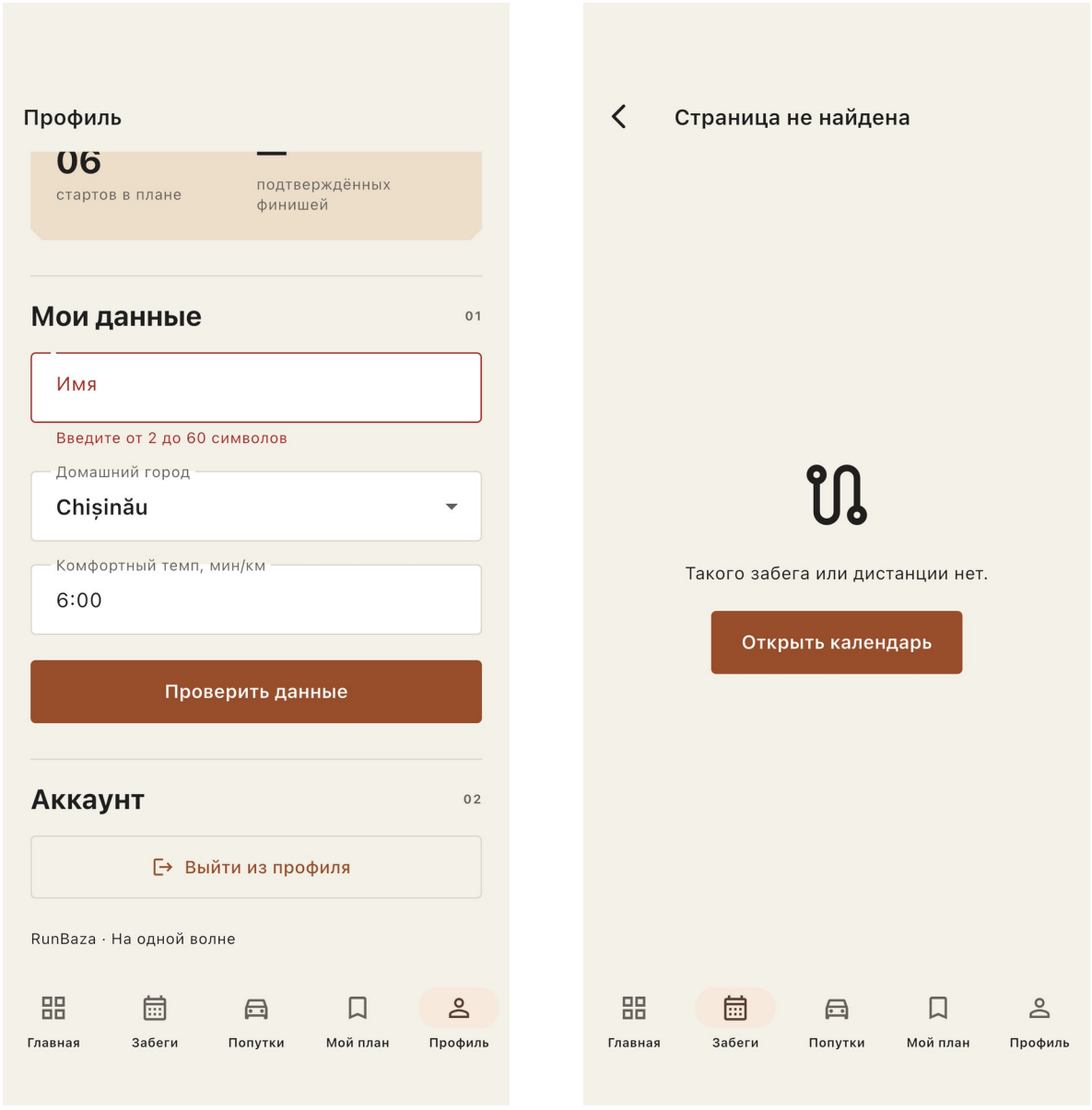



Рисунок 8. Валидация профиля и безопасный выход из неизвестного адреса.

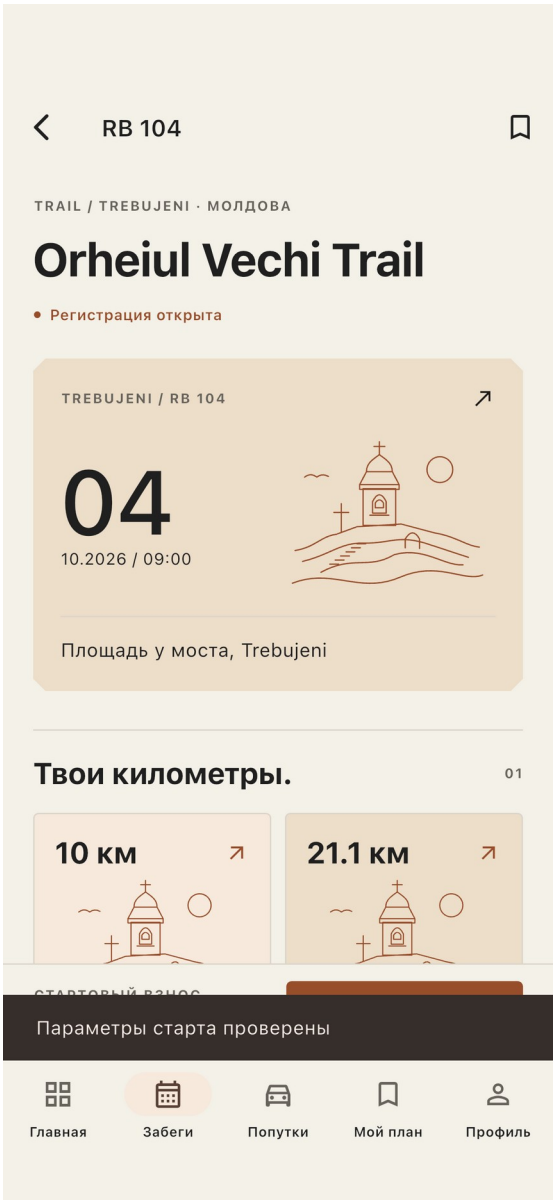
Кнопка календаря использует context.go и восстанавливает понятный маршрут. Проверки путей включают отсутствующий забег, неизвестную страницу, нечисловую и неподдерживаемую дистанцию.

# 12 Правила валидации

| Поле | Правило и сообщение | Клавиатура и фильтр |
|---|---|---|
| E-mail | Адрес без пробелов с @ и доменом; «Введите e-mail, например runner@example.com» | emailAddress |
| Пароль | Минимум 6 символов; «Не менее 6 символов» | Скрытый ввод; подсказки отключены |
| Имя и место встречи | 2–60 символов после trim; «Введите от 2 до 60 символов» | Текст |
| Город | Обязательный выбор; «Выберите город выезда» / «Выберите город» | DropdownButtonFormField |
| Время | ЧЧ:ММ, 00:00–23:59; «Время в формате ЧЧ:ММ, например 06:30» | datetime; цифры и :; до 5 знаков |
| Места | Целое 1–8; «От 1 до 8 мест» | number; только цифры; 1 знак |
| Вклад | Целое 0–5000; «От 0 до 5000 MDL» | number; только цифры; до 4 знаков |
| Телефон | 8–15 цифр, необязательный + в начале; «Введите 8–15 цифр, можно с + в начале» | phone; цифры и +; до 16 знаков |
| Дистанция | Из списка выбранного Race; «Выберите дистанцию» | DropdownButtonFormField |
| Темп | 3:00–15:00 мин/км; «Темп от 3:00 до 15:00 мин/км» | datetime; в форме участия цифры и : |
| Видимость | Обязательный выбор; «Выберите видимость» | Только мне / Участникам забега |

Validators — чистые функции в lib/forms/validators.dart. Корректное значение возвращает null, ошибка — текст. GlobalKey<FormState> создаётся один раз в State; validate проверяет все поля формы. Контроллеры создаются вне build и освобождаются в dispose. Все поля формы находятся в одной Column внутри ListView, поэтому остаются смонтированными и участвуют в validate даже за пределами экрана.

# 13 Сообщения и подтверждение выхода



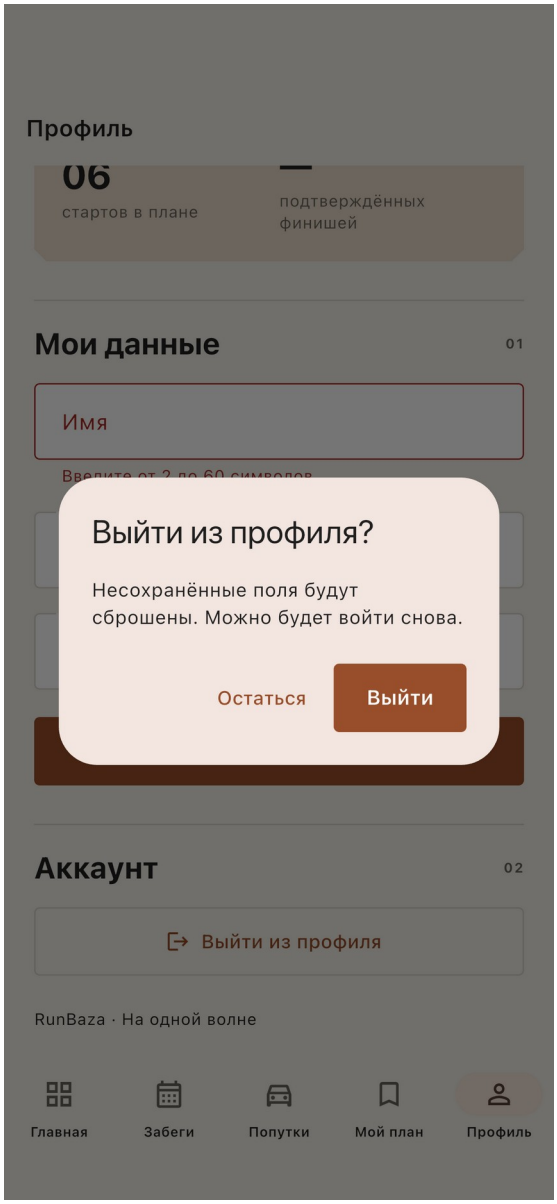




Рисунок 9. SnackBar после проверки формы и AlertDialog перед выходом.

finishForm скрывает клавиатуру, закрывает форму и показывает SnackBar. Выход выполняется только после ответа true из showDialog<bool>. «Остаться» возвращает false, закрытие окна касанием снаружи — null; оба варианта оставляют профиль открытым.

```
final confirmed = await showDialog<bool>(...);
if (!mounted || confirmed != true) return;
context.go('/login');
```

После await проверяется mounted, потому что экран мог быть удалён во время ожидания. Navigator.pop используется только для закрытия самого диалога с результатом; переходы между страницами выполняет go_router.

# 14 Проверка на iOS и автоматические тесты

Платформа проверки — iPhone 17 Pro Simulator, iOS 26.5. Снимки в отчёте получены из работающей iOS-сборки через integration_test. Сценарий проходит вход с ошибками и успешный вход, пять веток, разные ID, формы, SnackBar, отмену и подтверждение выхода.

```
flutter analyze --fatal-infos
flutter test
flutter build ios --simulator --debug --no-codesign
flutter drive --driver=test_driver/lab3_driver.dart \
  --target=integration_test/lab3_test.dart -d <simulator-id>
```

| Группа проверки | Что проверено |
|---|---|
| Валидация и навигация | Границы чисел и времени; успешная и ошибочная отправка; push/pop/go; отсутствие login в стеке |
| Независимые вкладки | Фильтр страны и поиск, сохранение позиции списка и вложенной карточки |
| Ошибочные адреса | Неизвестный ID, путь и дистанция без необработанного исключения |
| Адаптивность | 320/390/430 логических пикселей; светлая и тёмная темы; масштаб текста до 160% |
| Сохранённые функции Л2 | Анимация маршрута, повтор, Reduce Motion, география и целостность данных |

100 unit/widget-тестов завершились успешно. flutter analyze не выявил замечаний. Отдельный интеграционный сценарий проверяет iOS-приложение. GitHub Actions проверяет форматирование, анализ, тесты и сборку обеих лабораторных; ветка main сохраняет требование CI passed.

## Вывод

Статический интерфейс стал связным приложением с проверяемыми маршрутами и формами. Вкладки сохраняют локальное состояние, ошибочные данные не вызывают переход, а выход требует подтверждения. Для следующего этапа остаются сохранение, настоящая аутентификация, работа с API и обновление списков.

# 15 История Git и источники

Изменения выполнены в ветке lab/3-navigation-forms поверх Л2. Ниже история от исходного коммита Л2 до версии кода, использованной для подготовки отчёта. Коммит с документами добавляется после формирования этой страницы.

```
88d4d6c L3: сохранение всех полей длинной формы при прокрутке
cff1f74 L3: прокрутка длинных форм в сценарии iPhone
06fd54a L3: проверки навигации и форм, сценарий iOS и руководство запуска
5c0d665 L3: формы, валидация, профиль и подтверждение выхода
43dd389 L3: пять независимых вкладок StatefulShellRoute
ae86a65 L3: карта маршрутов и переходы по ID через go_router
0ec3823 L3: добавлен пакет go_router
df75976 Add Lab 2: RunBaza interface and animated distance screens
```

## Основные файлы

```
Lab2/run_baza/lib/router.dart
Lab2/run_baza/lib/widgets/app_shell.dart
Lab2/run_baza/lib/forms/validators.dart
Lab2/run_baza/lib/screens/*_form_screen.dart
Lab2/run_baza/test/navigation_forms_test.dart
Lab2/run_baza/integration_test/lab3_test.dart
Lab3/evidence/ios/
Lab3/Reports/
```

## Источники


L3_Navigare_formulare_validare_RU (1).pdf — задание лабораторной работы 3.

https://github.com/HamuevOleg/PAM — тема и исходный Flutter-проект.

https://github.com/dmitriaigenkiv/samepace — визуальная и продуктовая основа, main 438e942, изучен 11.10.2026.

https://pub.dev/documentation/go_router/latest/topics/Navigation-topic.html — push, pop и go.

https://pub.dev/documentation/go_router/latest/go_router/StatefulShellRoute-class.html — независимые стеки веток.

https://docs.flutter.dev/cookbook/forms/validation — Form, ключ и validator.